A Statistical Analysis of Beer Pong from Monte Carlo Simulations

Neil Pandya

2020
January

Contents

Abstract

In the interest of understanding the role of randomness and probability in Beer Pong, a Monte Carlo simulation was written and used. The simulation is purely probabilistic in the sense that the players do not aim, but instead throw balls at random locations on the opponent's side. The main hypothesis of this work is that, discounting any skill involved, the proportion of games which contain a big lead during the mid game then later end with a small lead is larger than the proportion of games which contain a big lead during the mid game then later end with a similarly large lead. The results show that the

hypothesis is confirmed. It was found that the number of times a ball lands in a cup as a function of time is a non-Markovian stochastic process, but may be modeled as a Markovian non-homogeneous Poisson process. Many statistics were gathered from the simulation such as the proportion of games won by the player to go first, the proportions of the different types of shots that can be made, the proportions of the different types of games that can be played, etc. The expected-score curve achieved $R^2 = 0.98$ when compared with the mean score of 3,000 simulated rack clearances.

1 Introduction

Last year during the holiday season, I found myself at an "ugly sweater" party. I came across two party-goers playing the classic party game, "Beer Pong." For those who are not familiar, to play the game in its simplest form, two players stand at opposite ends of a table while 10 red plastic cups are arranged in a triangle on both ends of the table such that–in the point of view of a player standing behind the table–4 cups are aligned horizontally nearest the edge of the table, 3 cups behind them, 2 cups behind those, and 1 cup behind those. See Figure 1. The cups are half-filled with beer, and players take turns throwing ping pong balls into the cups nearest their opponent, hence the name of the game. If a shot misses, it is the opponent's turn, but if the ball goes into a cup, the player goes again. The opponent also removes a cup from their side and drinks the beer inside. The game is won by the player who is first to make 10 shots. There are many more rules and variations to the game, but at its heart, the game is played in this way.

At this party, when I first viewed the table of two players mid-game, one of the players had a strong lead, to wit she had significantly more cups on her side than her opponent. After making this initial observation, I moseyed into the kitchen to fix myself a drink. Upon my return, I found the score to be tied with one cup on either side of the table. I asked the player who was initially in the lead what had happened, and her response birthed the analysis before you. She answered my question with her own question: "Haven't you ever heard of choking?" What she had implied was that when there are a small number of cups on the opponent's side, a psychological pressure affects the player's ability to perform well. The concept of "choking" under such pressure is very real and deep involving the idea of over-thinking as opposed to using the muscle memory developed from training and experience. Choking was certainly a possible explanation for the phenomenon which had occurred, but my hunch was of an orthogonal kind. I had figured that these types of games occur quite frequently due to the nature of random chance and probability, and not necessarily because a skilled player chokes at the end of a game. As the number of cups diminish on the opponent's side, so too does the probability of making a shot. This gives the opponent time to catch up and tie (or almost tie) the game at some point later. I posit the following hypothesis: there are significantly more games with large discrepancies in score at the beginning but small discrepancies in score at the end than there are games with large discrepancies in score at the beginning and large discrepancies in score at the end due to random chance. To prove this, and to expose many other statistics of the game, I have written software that simulates playthroughs of Beer Pong.

Keep in mind, a computer is not capable of choking. If, instead of simulations, a large number of real-life games were to be observed, and it was found that many games that started with large discrepancy scores ended with small discrepancy scores, then it would certainly support the idea of choking due to the fact that skills vary amongst players. However, if the skills of both players are equal and this phenomenon still occurred then probability would be left as the only explanation. In other words, if the condition that one player is more skilled than another player is not implied then the phenomenon of a small discrepancy final score given a large discrepancy score near the beginning of the game would still occur more frequently than the phenomenon of a large discrepancy final score given a large discrepancy score near the beginning of the game, and this phenomenon would be explained by randomness alone and not necessarily through a deterministic factor such as choking.

Figure 1: A typical setup of Beer Pong.

2 Design

2.1 Simulation

2.1.1 Differences and similarities between real-life games and simulated games

The simulation has key differences between its design and a typical real-life game of Beer Pong. The differences do not hinder the confirmation or denial of the hypothesis that games frequently end in scores that are close to each other due to random chance and not necessarily any other exogenous sources. The first difference is that each player begins with 6 cups instead of 10. This was implemented for the sake of the speed of the simulation. Ten cups would theoretically not change the results that are germane to the hypothesis, but at least six cups are necessary due to the fact that the next largest amount is three cups wherein games that end in large discrepancy scores are virtually nonexistent. Another main difference is that players do not aim at cups in any way; the players make random shots at the opponent's cups. This makes the game purely probabilistic. This does not hinder the confirmation or denial of the hypothesis because although games in real life are not purely probabilistic, they are highly probabilistic. The reason being that the aim of players is not perfect due to lack of skill, and external forces unknown or unaccounted for by the player cause indeterministic fluctuations. A huge probabilistic component of the game is the fact that when there are more cups on the opponent's side, balls that were not headed straight for the inside of a cup sometimes bounce off of the rim of a cup and land in the cup or a nearby cup. This is one of the reasons that the probability of getting a ball in a cup when there are fewer cups diminishes. This component was

incorporated into the design of the simulation.

Another difference between a real-life game and a simulated game is the game mechanic known as "re-racking." Many play Beer Pong with this rule, but it is absent in the simulation. This rule indicates that players at any point in the game may rearrange the cups on the opponent's side to the player's liking. Typically, players draw cups in closer to each other as removed cups have left space in between the remaining cups. This is usually allowed a maximum of twice per player per game.

There were many other rules/mechanics in Beer Pong that were not incorporated into the simulation, with which some players play and some do not. The following is a non-exhaustive list of these:

- If a player bounces a ball on the table during their turn and the ball lands in the opponent's cup, the opponent then removes a cup of the player's choosing in addition to the cup that had the ball land in it. However, if the player bounces the ball on the table, the opponent may swat the ball to prevent the shot from being made.
- If a ball spins around inside a cup, the opponent may blow the ball out of the cup or use their fingers to draw the ball out of the cup depending on whether the opponent is male or female, respectively.
- Players may play with two balls at a time instead of just one.
- Players may play two vs two. At parties, this is usually the case.
- Players may call in another player to substitute them for one turn. This is known as a "celebrity shot."
- If a ball bounces back to a player, the player may take another shot, but it must be from behind his/her back.
- In games of two vs two, if there is only one cup on the opponent's side both players on the opposite side must make the final shot in one round.

As mentioned earlier, a component of the game with probability built into its indeterministic nature was that a ball may hit the rim of a cup and land in the cup or a nearby cup. The simulation is designed such that if a ball hits the rim of a cup, it has an equal probability of bouncing one space left, right, forward, or back. This space may be a cup, slightly increasing the probability of making a shot. Also, the game mechanic of a player going again on the condition that he/she made a shot was incorporated into the simulations. This was added in the interest of knowing whether or not going first played a significant role in the results of the games.

2.1.2 Mechanics of simulated games

Both players start with a 9×9 grid on their side of the table. See Figure 2. Blue squares represent 6 cups and their placement on the grid. The arrangement is viewed by a player on the opposite side of the table who would be the one to throw balls at this grid. Each grid square represents a point that a ball can hit, each having probability $1/81$ of being hit on any given throw of a ball. Each point is one of three possible types: "cup," "no cup," or "rim." If a ball lands on a "no cup" square, it becomes the other player's turn to go. If it lands on a "rim" square, the ball randomly moves up, down, left, or right, each with a probability of $1/4$. The square it then bounces on may be any of the three types of squares. Finally, if the ball lands on a "cup" square, the player goes again, the cup is removed, and the player earns a point.

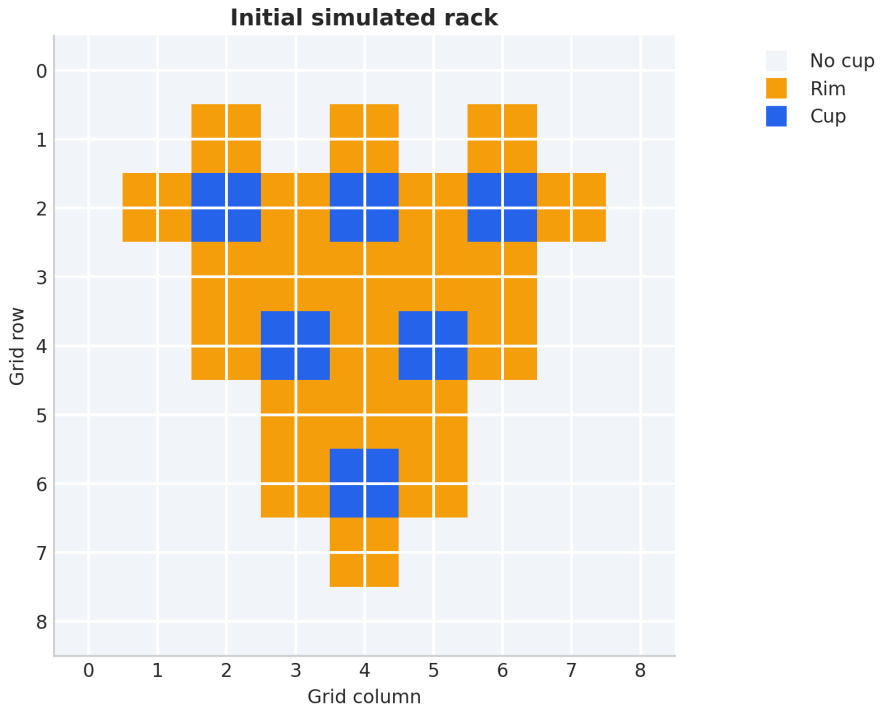

Figure 2: Theoretical arrangement of cups at the beginning of a game as viewed by a player on the opposite side.

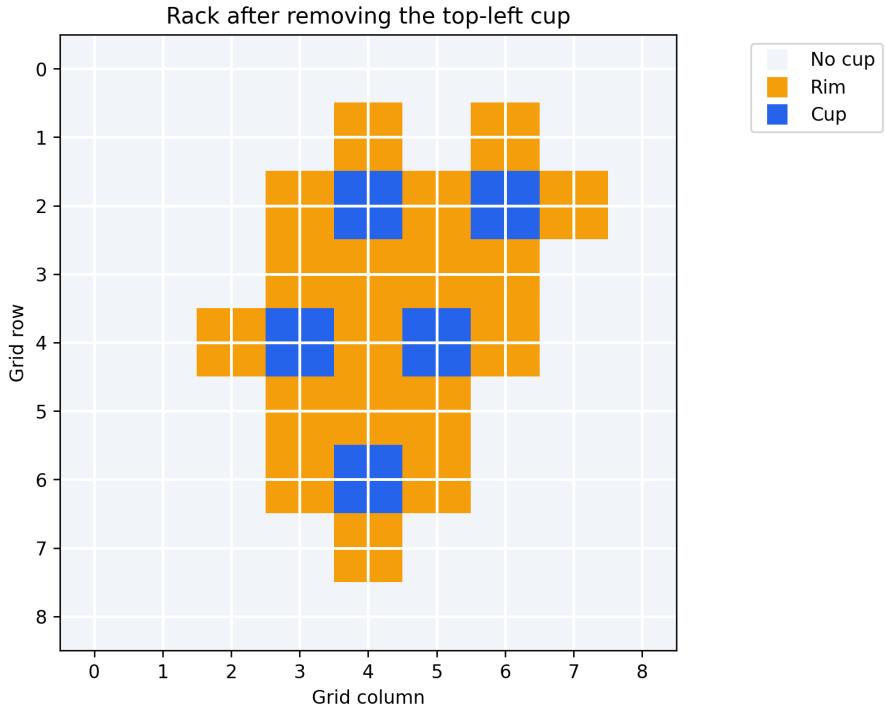

Figure 3: The appearance of the opponent's array after only the top-left cup has been removed. The rim shared with the cup to its right remains.

When a cup is removed, the "cup" square changes to "no cup" as well as the "rim" squares above and below it, and sometimes to the left and right depending on which cup is removed and whether or not there is a cup to its left or right. The reason it depends on these conditions is because cups to the left and right of one another share the same rim. For example, if the cup at the far left and very top were to be removed as in Figure 3, only the "rim" squares above, below, and to the left change to "no cup" squares because the cup to the right has not yet been removed and so it must keep its "rim" square. If, for example, the very top cup in the center, i.e. the cup directly to the right of the cup that was just removed, were now removed, the "rim" square to its left may now be removed.

The reader at this point may wonder about the size of the border surrounding the three cups. The border is necessary due to the fact that real-life players often miss the triangle of cups entirely. The size of the border is only one square because the players in the simulations are already not aiming at all, and so each game would take too long if the size were larger.

There are four types of shots that can be made. The first is hitting a rim of a cup, and then (eventually) landing the ball outside of any cup, i.e. into a "no cup" square. The next type of shot is directly into a "no cup" square, i.e. without hitting a rim first. The third type of shot is hitting a rim, and then (eventually) landing the ball inside a cup. The last type of shot is landing the ball directly inside a cup without first hitting the rim. In the interest of which types of shots were more common than others, the simulator keeps count of the types of shots in a single game.

2.2 Software

Understanding exactly how the software works and all of its logic is vital to adjudicating the validity of the simulation and veracity of the conclusions drawn from it. The code below was used to produce the results in this paper.

2.2.1 Libraries

The code was written in Python 3 using Jupyter. NumPy is used for arrays, random number generation, and mathematical functions; pandas is used to gather the results into tables; and matplotlib is used to plot the figures. The code also uses a few tools from Python's standard library to keep count of shot types, define the objects used by the simulator, and calculate factorials.

```
from __future__ import annotations

from collections import Counter
from dataclasses import dataclass
from math import factorial
from typing import Iterable

import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
import numpy as np
import pandas as pd

plt.style.use("seaborn-v0_8-whitegrid")
plt.rcParams.update({
    "figure.figsize": (9, 5),
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.titleweight": "bold",
    "figure.dpi": 120,
})

SEED = 2020
GRID_SIZE = 9
WINNING_SCORE = 6

MISS, RIM, CUP = 0, 1, 2
DIRECTIONS = np.array([(0, -1), (-1, 0), (0, 1), (1, 0)])

CUP_COORDINATES = (
    (2, 2), (2, 4), (2, 6),
    (4, 3), (4, 5),
    (6, 4),
)
```

```
RIM_COORDINATES = (
    (1, 2), (1, 4), (1, 6),
    (2, 1), (2, 3), (2, 5), (2, 7),
    (3, 2), (3, 3), (3, 4), (3, 5), (3, 6),
    (4, 2), (4, 4), (4, 6),
    (5, 3), (5, 4), (5, 5),
    (6, 3), (6, 5),
    (7, 4),
)
```

2.2.2 Setting up a game

One of the functions written performs the simple job of creating an array for each player. The same function is used to set up the same array at the beginning of each game for both players, producing identical setups. Each element in the 9×9 grid is given an integer value representing "no cup," "rim," or "cup." The `Rack` object keeps track of the grid, counts the cups that remain, and removes the rim squares that disappear when a cup is made. When the cup at $(4, 3)$ is removed, the shared rim at $(4, 4)$ remains only if the cup at $(4, 5)$ is still present.

```
def initial_grid() -> np.ndarray:
    '''Return the paper's 9x9 six-cup rack.'''
    grid = np.full((GRID_SIZE, GRID_SIZE), MISS, dtype=np.int8)
    for coordinate in RIM_COORDINATES:
        grid[coordinate] = RIM
    for coordinate in CUP_COORDINATES:
        grid[coordinate] = CUP
    return grid


@dataclass
class Rack:
    '''A mutable target rack belonging to one player.'''
    def __post_init__(self) -> None:
        self.grid = initial_grid()

    @property
    def cups_remaining(self) -> int:
        return int(np.count_nonzero(self.grid == CUP))

    def remove_cup(self, row: int, column: int) -> None:
        '''Remove a made cup and the rim cells no longer supported by cups.'''
        if self.grid[row, column] != CUP:
            raise ValueError(f"No cup exists at {(row, column)}")

        self.grid[row, column] = MISS
        self.grid[row - 1, column] = MISS
        self.grid[row + 1, column] = MISS

        if (row, column) == (2, 2):
            self.grid[2, 1] = MISS
            if self.grid[2, 4] == MISS:
                self.grid[2, 3] = MISS
        elif (row, column) == (2, 4):
            if self.grid[2, 2] == MISS:
                self.grid[2, 3] = MISS
            if self.grid[2, 6] == MISS:
                self.grid[2, 5] = MISS
        elif (row, column) == (2, 6):
            self.grid[2, 7] = MISS
            if self.grid[2, 4] == MISS:
                self.grid[2, 5] = MISS
        elif (row, column) == (4, 3):
            self.grid[4, 2] = MISS
            if self.grid[4, 5] == MISS:
                self.grid[4, 4] = MISS
        elif (row, column) == (4, 5):
            self.grid[4, 6] = MISS
            if self.grid[4, 3] == MISS:
                self.grid[4, 4] = MISS
        elif (row, column) == (6, 4):
            self.grid[6, 3] = MISS
            self.grid[6, 5] = MISS


@dataclass(frozen=True)
```

```
class ShotResult:
    scored: bool
    kind: str
    start: tuple[int, int]
    end: tuple[int, int]
    bounces: int


@dataclass(frozen=True)
class ThrowRecord:
    global_throw: int
    player: int
    player_attempt: int
    shot_kind: str
    scored: bool
    player1_score: int
    player2_score: int


@dataclass
class GameResult:
    first_player: int
    winner: int
    attempts: tuple[int, int]
    event_times: tuple[tuple[int, ...], tuple[int, ...]]
    records: tuple[ThrowRecord, ...]

    @property
    def final_scores(self) -> tuple[int, int]:
        return (
            self.records[-1].player1_score,
            self.records[-1].player2_score,
        )

    @property
    def score_differences(self) -> np.ndarray:
        return np.array(
            [0] + [r.player1_score - r.player2_score for r in self.records]
        )

    @property
    def max_absolute_lead(self) -> int:
        return int(np.abs(self.score_differences).max())

    @property
    def final_margin(self) -> int:
        return int(abs(self.final_scores[0] - self.final_scores[1]))

    @property
    def game_type(self) -> str:
        if self.max_absolute_lead <= 3:
            return "Neck and neck"
        if self.final_margin <= 3:
            return "Apparent choke"
        return "Landslide"

    @property
    def signed_biggest_lead(self) -> int:
        differences = self.score_differences
        positive = int(differences.max())
        negative = int(differences.min())
        if positive > abs(negative):
            return positive
        if abs(negative) > positive:
            return negative
        return 0

    @property
    def surprise_type(self) -> str:
        lead = self.signed_biggest_lead
        if lead == 0:
            return "Other"
        leader = 1 if lead > 0 else 2
        return "No surprise" if leader == self.winner else "Surprise"

    def history_frame(self) -> pd.DataFrame:
        return pd.DataFrame([record.__dict__ for record in self.records])
```

2.2.3 Playing 1 game

The next function plays 1 game of Beer Pong. The variable it takes in is either the integer number 1 or 2, indicating which player goes first. The function sets up the two player arrays, initializes the players' scores and attempts, and keeps throwing balls until one of the players has scored 6 points. It returns a `GameResult` containing the winner, the event times of both players, and a record of every throw.

```
def simulate_shot(rack: Rack, rng: np.random.Generator) -> ShotResult:
    '''Simulate one throw at a rack and mutate the rack after a make.'''
    row, column = map(int, rng.integers(0, GRID_SIZE, size=2))
    start = (row, column)
    hit_rim = rack.grid[row, column] == RIM
    bounces = 0

    while rack.grid[row, column] == RIM:
        delta_row, delta_column = DIRECTIONS[int(rng.integers(0, 4))]
        row += int(delta_row)
        column += int(delta_column)
        bounces += 1
        if bounces > 10_000:
            raise RuntimeError("Rim random walk did not terminate")

    scored = rack.grid[row, column] == CUP
    if scored:
        rack.remove_cup(row, column)

    if hit_rim and scored:
        kind = "Rim, then cup"
    elif hit_rim:
        kind = "Rim, then miss"
    elif scored:
        kind = "Direct cup"
    else:
        kind = "Direct miss"

    return ShotResult(scored, kind, start, (row, column), bounces)


def simulate_game(
    first_player: int = 1,
    *,
    rng: np.random.Generator | None = None,
) -> GameResult:
    '''Simulate one complete two-player game.'''
    if first_player not in (1, 2):
        raise ValueError("first_player must be 1 or 2")
    rng = np.random.default_rng() if rng is None else rng

    racks = [Rack(), Rack()]
    scores = [0, 0]
    attempts = [0, 0]
    event_times: list[list[int]] = [[], []]
    records: list[ThrowRecord] = []
    player = first_player - 1

    while max(scores) < WINNING_SCORE:
        attempts[player] += 1
        target_rack = racks[1 - player]
        shot = simulate_shot(target_rack, rng)

        if shot.scored:
            scores[player] += 1
            event_times[player].append(attempts[player])

        records.append(
            ThrowRecord(
                global_throw=len(records) + 1,
                player=player + 1,
                player_attempt=attempts[player],
                shot_kind=shot.kind,
                scored=shot.scored,
                player1_score=scores[0],
                player2_score=scores[1],
            )
        )

        if not shot.scored:
            player = 1 - player
```

```
    winner = 1 if scores[0] == WINNING_SCORE else 2
    return GameResult(
        first_player=first_player,
        winner=winner,
        attempts=tuple(attempts),
        event_times=(tuple(event_times[0]), tuple(event_times[1])),
        records=tuple(records),
    )
```

2.2.4 Throwing the ball

The player chooses a random row and a random column of the opposing player's array and throws the ball at that position. If the ball hits a rim, it moves up, down, left, or right with equal probability. In some circumstances, the ball hits another rim and stays in the while loop until it moves to a "cup" or a "no cup" square. If the ball hits a cup, the code keeps track of whether the shot had hit a rim first, adds a point to the player's score, removes the cup, and allows the same player to throw again. This logic is contained in `simulate_shot` and is used identically for both players.

The simulator also performs a few smoke tests. These are quick tests which check that the rack begins with six cups and the right number of rim squares, that a simulated game has a winner with six points, and that every recorded shot belongs to one of the four possible types.

```
def plot_rack(rack: Rack, title: str = "Initial simulated rack") -> None:
    cmap = ListedColormap(["#f1f5f9", "#f59e0b", "#2563eb"])
    fig, ax = plt.subplots(figsize=(6, 6))
    ax.imshow(rack.grid, cmap=cmap, vmin=MISS, vmax=CUP)
    ax.set(
        title=title,
        xlabel="Grid column",
        ylabel="Grid row",
        xticks=range(GRID_SIZE),
        yticks=range(GRID_SIZE),
    )
    ax.grid(color="white", linewidth=1.5)
    legend = [
        plt.Line2D([0], [0], marker="s", color="w", markerfacecolor=color,
                   markersize=12, label=label)
        for color, label in [
            ("#f1f5f9", "No cup"),
            ("#f59e0b", "Rim"),
            ("#2563eb", "Cup"),
        ]
    ]
    ax.legend(handles=legend, loc="upper right", bbox_to_anchor=(1.35, 1.0))
    plt.show()


rack = Rack()
assert rack.cups_remaining == WINNING_SCORE
assert np.count_nonzero(rack.grid == RIM) == len(RIM_COORDINATES)

smoke_test = simulate_game(first_player=1, rng=np.random.default_rng(SEED))
assert WINNING_SCORE in smoke_test.final_scores
assert len(smoke_test.event_times[smoke_test.winner - 1]) == WINNING_SCORE
assert set(r.shot_kind for r in smoke_test.records) <= {
    "Direct miss", "Direct cup", "Rim, then miss", "Rim, then cup"
}

plot_rack(rack)
```

2.2.5 Playing many games

This function takes an integer variable which tells it how many games to play. The statistics obtained from playing 1 game are gathered here by playing many games. The results from 10,000 games are kept in one tidy dataframe wherein every row represents one game. This is mathematically equivalent to first dividing the games into equally sized sets and then averaging the set means.

Overall there are three types of games that can occur: apparent chokes, landslides, and neck and neck games. Although the term "chokes" is used here, they are not games where a computer "chokes" and feels human psychological pressure. They are games that appear to be chokes in the probabilistic sense. More

specifically, they are games which contain high discrepancy scores but end with low discrepancy scores. The program constitutes games with maximum score differences at any point in the game larger than 3 and with final score differences smaller than or equal to 3 as “apparent chokes.” Landslides have a maximum and final score difference greater than 3, while neck and neck games never have a score difference greater than 3.

The types of games that are played may be categorized in a different way as well, i.e. into “surprises,” “no surprises,” and “others.” Games which are surprises are those where the player with the biggest lead did not turn out to be the winner. No surprises are games where the player with the biggest lead did win. Finally, there are “others.” These games are given to those where both players at some time held equally large leads.

```
def simulate_many_games(
    n_games: int,
    *,
    seed: int = SEED,
    first_player: int | None = None,
) -> pd.DataFrame:
    '''Return one tidy summary row per simulated game.'''
    rng = np.random.default_rng(seed)
    rows = []

    for game_id in range(n_games):
        starter = int(rng.integers(1, 3)) if first_player is None else first_player
        result = simulate_game(starter, rng=rng)
        shot_counts = Counter(record.shot_kind for record in result.records)
        n_throws = len(result.records)

        rows.append({
            "game_id": game_id,
            "first_player": starter,
            "winner": result.winner,
            "player1_score": result.final_scores[0],
            "player2_score": result.final_scores[1],
            "total_throws": n_throws,
            "player1_attempts": result.attempts[0],
            "player2_attempts": result.attempts[1],
            "attempt_difference": result.attempts[0] - result.attempts[1],
            "max_absolute_lead": result.max_absolute_lead,
            "final_margin": result.final_margin,
            "game_type": result.game_type,
            "surprise_type": result.surprise_type,
            **{
                f"share_{kind.lower().replace(', ', '_').replace(' ', '_')}":
                    shot_counts[kind] / n_throws
                for kind in (
                    "Direct miss", "Direct cup",
                    "Rim, then miss", "Rim, then cup",
                )
            },
        })

    return pd.DataFrame(rows)


games = simulate_many_games(10_000, seed=SEED)
games.head()

def proportion_table(series: pd.Series, order: Iterable[str]) -> pd.DataFrame:
    proportions = series.value_counts(normalize=True).reindex(order, fill_value=0.0)
    return proportions.rename("proportion").to_frame().assign(
        percent=lambda frame: 100 * frame["proportion"]
    )


game_types = proportion_table(
    games["game_type"],
    ["Neck and neck", "Landslide", "Apparent choke"],
)
surprise_types = proportion_table(
    games["surprise_type"],
    ["Surprise", "No surprise", "Other"],
)

fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
for ax, table, title, color in [
    (axes[0], game_types, "Game types I", "#2563eb"),
    (axes[1], surprise_types, "Game types II", "#f97316"),
```

```
]:
    bars = ax.bar(table.index, table["percent"], color=color, alpha=0.85)
    ax.bar_label(bars, fmt="%.1f%%", padding=3)
    ax.set(title=title, ylabel="Share of simulated games (%)")
    ax.tick_params(axis="x", rotation=18)
    ax.set_ylim(0, max(table["percent"]) * 1.15)
plt.tight_layout()
plt.show()

summary = pd.Series({
    "Player 1 win rate": (games["winner"] == 1).mean(),
    "Mean final margin": games["final_margin"].mean(),
    "Mean total throws": games["total_throws"].mean(),
    "P(apparent choke | lead > 3)": (
        (games["game_type"] == "Apparent choke").sum()
        / (games["max_absolute_lead"] > 3).sum()
    ),
})
display(summary.to_frame("estimate").style.format("{:.4f}"))
display(game_types.style.format({"proportion": "{:.4f}", "percent": "{:.2f}"}))
```

2.2.6 A single trajectory

The score of one player as a function of the number of times that player has thrown the ball is denoted by P_t. Since a player's score changes only when a cup is made, a trajectory of P_t is a step function.

```
sample_game = simulate_game(first_player=1, rng=np.random.default_rng(2042))
winner_index = sample_game.winner - 1
winner_events = np.array(sample_game.event_times[winner_index])
final_attempt = sample_game.attempts[winner_index]

t_sample = np.arange(final_attempt + 1)
score_sample = np.searchsorted(winner_events, t_sample, side="right")

fig, ax = plt.subplots()
ax.step(t_sample, score_sample, where="post", linewidth=2.5, color="#2563eb")
ax.scatter(winner_events, np.arange(1, WINNING_SCORE + 1), color="#dc2626", zorder=3)
ax.set(
    title=f"Player {sample_game.winner}'s score trajectory",
    xlabel="Attempts by the winning player, $t$",
    ylabel="Cumulative score, $P_t$",
    xlim=(0, final_attempt),
    ylim=(-0.1, WINNING_SCORE + 0.3),
    yticks=range(WINNING_SCORE + 1),
)
plt.show()

sample_game.history_frame().head(10)
```

2.2.7 Unconditional rack-clearing event times

To model P_t, it is necessary to know about when the first, second, third, etc. points are scored. A one-player version of the game was therefore created where one player throws balls at a rack until all six cups are cleared.

Each simulation produces six event times,

$$(T_1, T_2, \ldots, T_6),$$

where T_n is the number of attempts required to score the nth point. The function repeats this 10,000 times and takes the mean of each column. Because every rack clearance is kept, these are unconditional mean event times.

```
def simulate_rack_clearance(
    *,
    rng: np.random.Generator | None = None,
) -> tuple[int, ...]:
    """Throw at one rack until all six cups have been cleared."""
    rng = np.random.default_rng() if rng is None else rng
    rack = Rack()

    attempts = 0
```

```
    event_times = []

    while rack.cups_remaining > 0:
        attempts += 1
        shot = simulate_shot(rack, rng)

        if shot.scored:
            event_times.append(attempts)

    return tuple(event_times)

def collect_unconditional_event_times(
    n_runs: int,
    *,
    seed: int,
) -> np.ndarray:
    """Collect six event times from independent rack-clearance simulations."""
    rng = np.random.default_rng(seed)

    event_times = [
        simulate_rack_clearance(
            rng=rng,
        )
        for _ in range(n_runs)
    ]

    return np.asarray(event_times, dtype=int)

unconditional_events = collect_unconditional_event_times(
    10_000,
    seed=2101,
)

unconditional_mean_times = unconditional_events.mean(axis=0)

unconditional_mean_times
```

The resulting times are organized in Table 1.

Table 1: Unconditional mean event times from 10,000 rack-clearance simulations.

n	0	1	2	3	4	5	6
t_n	0	4.9052	11.2322	19.6792	31.7763	50.9177	90.8531

2.2.8 $\lambda(t)$

The functions above revealed the expected intervals of time that P_t increased by 1. Those times were 4.9052, 11.2322, 19.6792, 31.7763, 50.9177, and 90.8531. Since [1]

$$E[P_t] = m(t) = \int_0^t \lambda(s)\, ds, \tag{1}$$

this suggests that what might work as an approximation to $\lambda(t)$ is

$$\lambda_i \approx \frac{1}{t_i - t_{i-1}}, \qquad t_{i-1} < t \leq t_i. \tag{2}$$

The values are shown in Table 2. The first interval begins at $t_0 = 0$. The last interval runs from 50.9177 to 90.8531 attempts.

Table 2: The experimentally obtained stepwise intensity.

Interval	Interval width	λ_i
$(0, 4.9052]$	4.9052	0.203865
$(4.9052, 11.2322]$	6.3270	0.158053
$(11.2322, 19.6792]$	8.4470	0.118385
$(19.6792, 31.7763]$	12.0971	0.082664
$(31.7763, 50.9177]$	19.1414	0.052243
$(50.9177, 90.8531]$	39.9354	0.025040

The following function produces such an experimentally obtained $\lambda(t)$. It then samples the step function at 100 evenly spaced times so that a more analytical lambda can be obtained using these values and linear regression.

```
def stepwise_intensity(
    t: np.ndarray,
    event_time_boundaries: np.ndarray,
) -> np.ndarray:
    levels = 1.0 / np.diff(event_time_boundaries)
    interval = np.searchsorted(event_time_boundaries[1:], t, side="left")
    return levels[np.clip(interval, 0, len(levels) - 1)]

event_time_boundaries = np.concatenate([
    [0.0],
    unconditional_mean_times,
])
FIT_END = event_time_boundaries[-1]  # approximately 90.8531
PLOT_END = 100

t_fit = np.linspace(
    0,
    FIT_END,
    100,
)

lambda_step = stepwise_intensity(
    t_fit,
    event_time_boundaries,
)

decay_rate, log_scale = np.polyfit(t_fit, np.log(lambda_step), deg=1)
scale = float(np.exp(log_scale))

def smooth_intensity(t: np.ndarray | float) -> np.ndarray:
    return scale * np.exp(decay_rate * np.asarray(t))


t_plot = np.linspace(
    0,
    PLOT_END,
    500,
)

fig, axes = plt.subplots(1, 2, figsize=(12, 4.5), sharey=True)
axes[0].step(
    t_fit, lambda_step, where="post", linewidth=2.5, color="#2563eb"
)
axes[0].set(title="Empirical $\\lambda(t)$")
axes[1].plot(t_plot, smooth_intensity(t_plot), linewidth=2.5, color="#2563eb")
axes[1].set(title="Smooth $\\lambda(t)$")

for ax in axes:
    ax.set(xlabel="Player attempts, $t$", ylabel="Scoring intensity")
    ax.set_xlim(0, PLOT_END)
    ax.set_ylim(0, 0.25)
plt.tight_layout()
plt.show()

print(f"Fitted intensity: lambda(t) = {scale:.15f} * exp({decay_rate:.11f} t)")
```

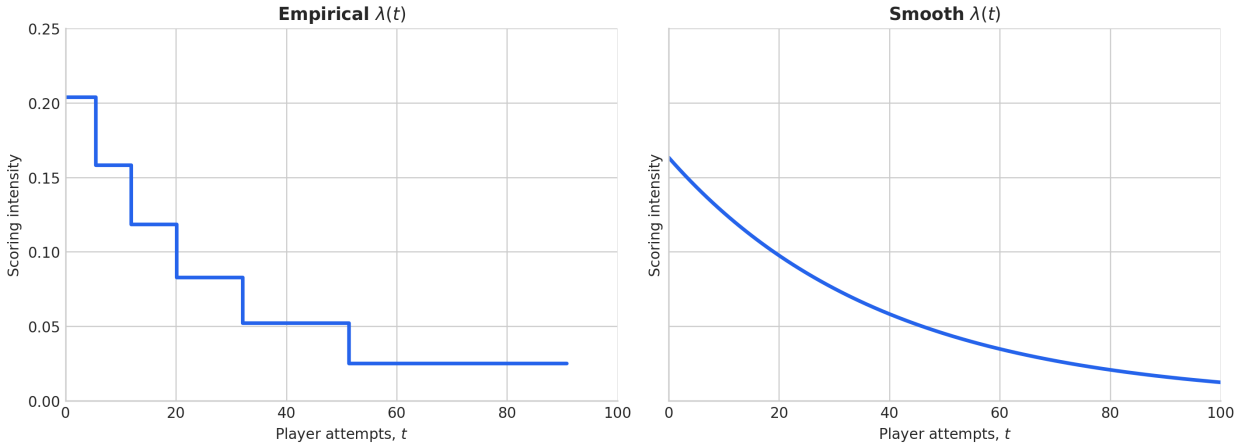

Figure 4: The experimentally found $\lambda(t)$ and the smooth function obtained from it.

2.2.9 Linear regression

It was assumed that $\lambda(t)$ takes on the form

$$\lambda(t) = Ce^{rt}. \tag{3}$$

To find C and r, linear regression was used by first using a different variable to linearize the model:

$$z = \log(\lambda) = \log(Ce^{rt}) \tag{4}$$
$$= \log(C) + rt. \tag{5}$$

The degree-one `numpy.polyfit` function performs this linear regression and returns the slope r and intercept $\log(C)$. The function returned 0.163349587480195 as C and -0.02578022137 as r, making

$$\lambda(t) = 0.163349587480195e^{-0.02578022137t}. \tag{6}$$

Substituting this into Eqn. 1 gives

$$E[P_t] = m(t) = \frac{0.163349587480195}{-0.02578022137}\left(e^{-0.02578022137t} - 1\right). \tag{7}$$

```
def expected_score(t: np.ndarray | float) -> np.ndarray:
    t = np.asarray(t)
    return (scale / decay_rate) * (np.exp(decay_rate * t) - 1.0)


fig, ax = plt.subplots()
ax.plot(t_plot, expected_score(t_plot), linewidth=3, color="#2563eb")
ax.set(
    title="Expected cumulative score",
    xlabel="Player attempts, $t$",
    ylabel="$m(t)=E[P_t]$",
    xlim=(0, PLOT_END),
    ylim=(0, 7),
)
plt.show()
```

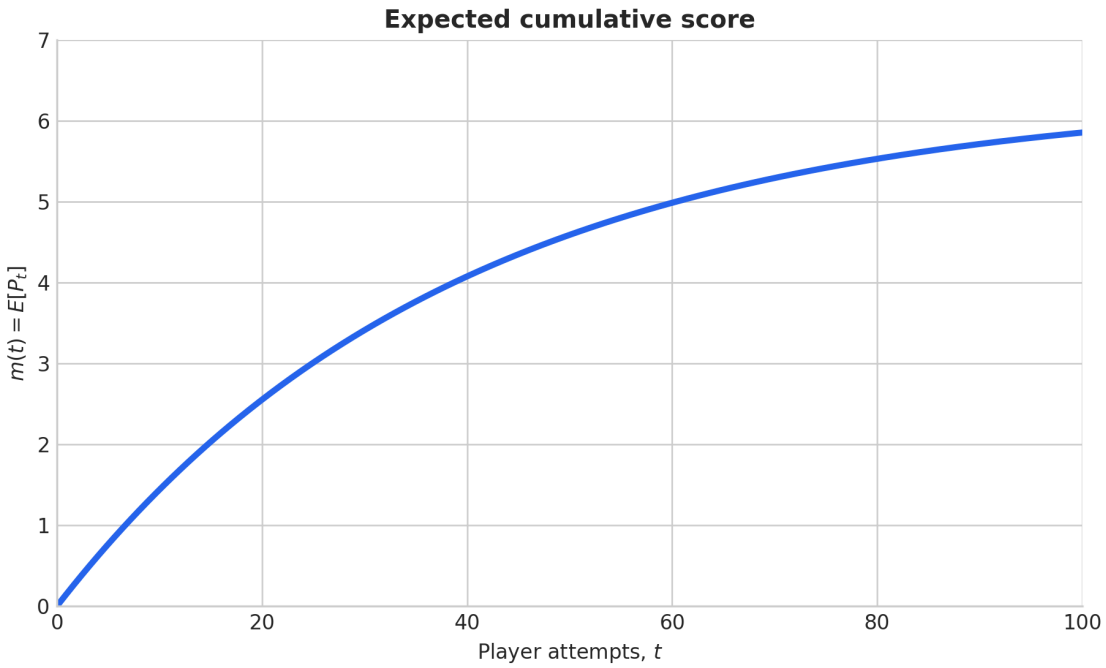

Figure 5: The expected number of points a player will earn as a function of the number of times that player throws the ball, $E[P_t] = m(t)$.

2.2.10 $P(P_t = n)$

A function can then be made to find the probability that $P_t = n$ for some time t, or number of times the player threw the ball. This probability is [1]

$$P(P_t = n) = \frac{[m(t)]^n}{n!} e^{-m(t)}. \tag{8}$$

```
def poisson_state_probability(t: np.ndarray, n: int) -> np.ndarray:
    mean = expected_score(t)
    return np.power(mean, n) * np.exp(-mean) / factorial(n)


fig, axes = plt.subplots(3, 2, figsize=(11, 11), sharex=True)
for n, ax in enumerate(axes.flat, start=1):
    ax.plot(t_plot, poisson_state_probability(t_plot, n),
            linewidth=2.5, color="#2563eb")
    ax.set(title=rf"$P(P_t={n})$", ylabel="Probability")
    ax.set_xlim(0, PLOT_END)
    ax.set_ylim(bottom=0)
for ax in axes[-1]:
    ax.set_xlabel("Player attempts, $t$")
fig.suptitle("Poisson state probabilities", fontsize=15)
plt.tight_layout()
plt.show()
```

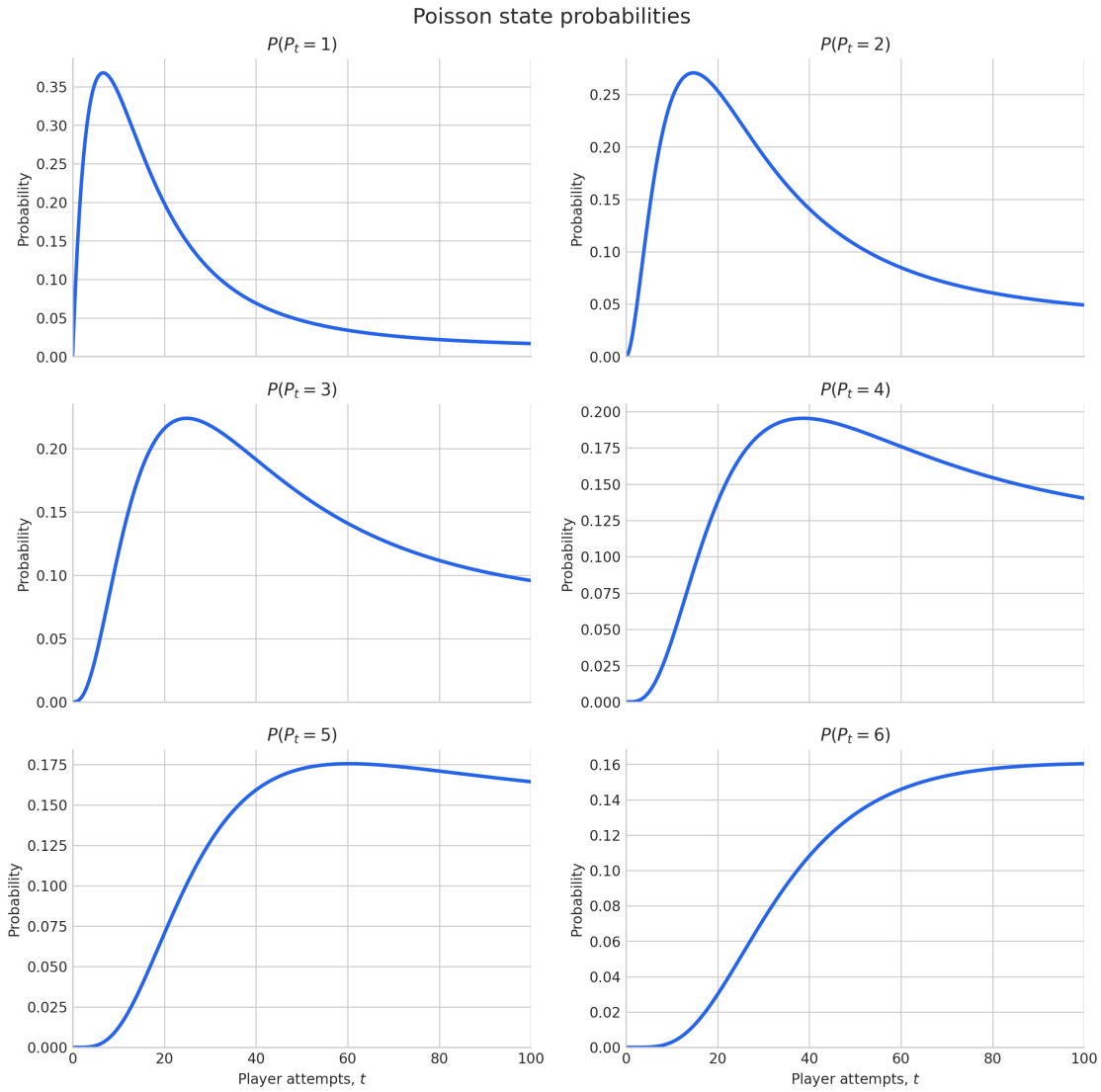

Figure 6: Probability distributions, $P(P_t = n)$, for each n.

2.2.11 Goodness of fit

Finally, 3,000 rack clearances were simulated. At each attempt, the mean simulated score was compared to $m(t)$. The code also calculates R^2, RMSE, and MAE, and plots the residuals.

```
# Generate new unconditional rack-clearance trajectories using a different seed.
holdout_events = collect_unconditional_event_times(
    3_000,
    seed=2202,
)

# Evaluate every integer attempt across the plotting range.
t_holdout = np.arange(0, int(PLOT_END) + 1)

# For each simulated rack and time t, the score is the number of
# scoring-event times less than or equal to t.
holdout_scores = (
    holdout_events[:, :, None] <= t_holdout[None, None, :]
).sum(axis=1)

empirical_mean = holdout_scores.mean(axis=0)
```

```
standard_error = (
    holdout_scores.std(axis=0, ddof=1)
    / np.sqrt(len(holdout_scores))
)

predicted_mean = expected_score(t_holdout)
residuals = empirical_mean - predicted_mean


# Goodness-of-fit statistics
ss_residual = np.sum(np.square(residuals))
ss_total = np.sum(
    np.square(empirical_mean - empirical_mean.mean())
)

r_squared = 1.0 - ss_residual / ss_total
rmse = float(np.sqrt(np.mean(np.square(residuals))))
mae = float(np.mean(np.abs(residuals)))
max_error = float(np.max(np.abs(residuals)))


# Plot the fitted model, holdout mean, confidence interval, and residuals.
fig, axes = plt.subplots(
    2,
    1,
    figsize=(10, 8),
    sharex=True,
    gridspec_kw={"height_ratios": [3, 1]},
)

axes[0].plot(
    t_holdout,
    predicted_mean,
    linewidth=3,
    color="#dc2626",
    label="Fitted expected-score model",
)

axes[0].plot(
    t_holdout,
    empirical_mean,
    linewidth=2,
    color="#2563eb",
    label="Held-out unconditional mean",
)

axes[0].fill_between(
    t_holdout,
    empirical_mean - 1.96 * standard_error,
    empirical_mean + 1.96 * standard_error,
    color="#2563eb",
    alpha=0.18,
    label="Approx. 95% confidence interval",
)

axes[0].set(
    ylabel="Mean cumulative score",
    title="Held-out fit to unconditional rack clearances",
)

axes[0].set_xlim(0, PLOT_END)
axes[0].set_ylim(bottom=0)
axes[0].legend()


axes[1].axhline(
    0,
    color="black",
    linewidth=1,
)

axes[1].plot(
    t_holdout,
    residuals,
    color="#7c3aed",
    linewidth=2,
)

axes[1].set(
    xlabel="Player attempts, $t$",
    ylabel="Residual",
```

```
)

axes[1].set_xlim(0, PLOT_END)

plt.tight_layout()
plt.show()


goodness_of_fit = pd.Series({
    "R-squared": r_squared,
    "RMSE (points)": rmse,
    "MAE (points)": mae,
    "Maximum absolute error (points)": max_error,
    "Holdout rack clearances": len(holdout_events),
})

display(
    goodness_of_fit
    .to_frame("value")
    .style.format("{:.6f}")
)
```

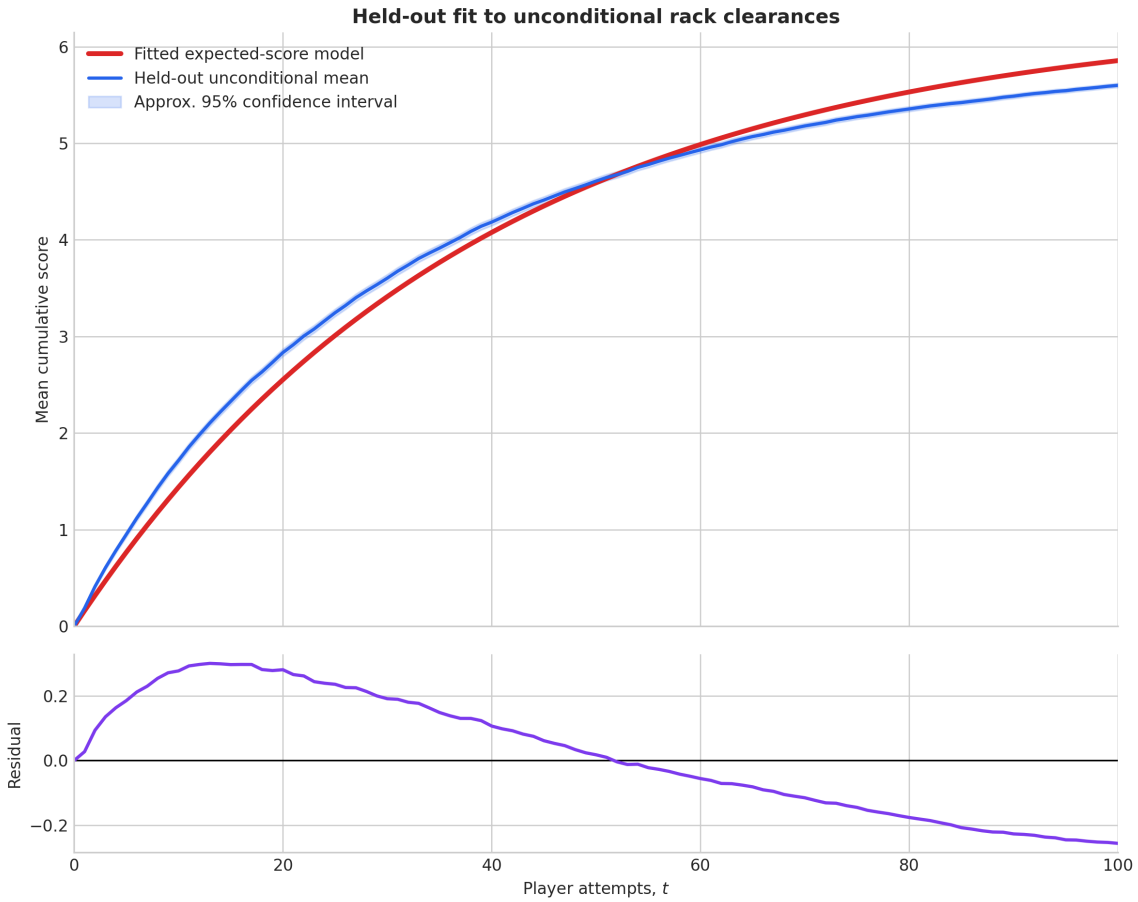

Figure 7: The fitted expected-score curve compared with the mean of 3,000 rack-clearance simulations.

3 Results

3.1 Significance of going first

It is of interest to know whether or not throwing the ball first works in favor of or against the player that does it. In the experiment, which player goes first was chosen at random in each of 10,000 games. The player

who went first won 50.01% of the games. It is therefore inconclusive if going first plays any contribution to winning a game of Beer Pong. As a consequence, the statistics gathered in the remainder of the paper were gathered from simulations of games without regard to which player went first.

3.2 Differences in final scores

The distribution of final scores was also of interest because it may provide insight into the hypothesis of the paper. The mean absolute difference in the scores of the two players on the very last round was 1.6371. The results show that a typical (simulated) game of Beer Pong ends with the winning player having about 1 to 2 cups more than the losing player. Assuming a spectator had only this data to reference, after observing the beginning of a real-life game where the player in the lead still had many more cups on his/her side than his/her opponent, the safest bet to make on the amount of cups with which that player ends the game is about 2.

Table 3: Distribution of final score differences in 10,000 simulated games.

Difference	Proportion	Difference	Proportion	Difference	Proportion
1	57.47%	2	27.26%	3	10.70%
4	3.48%	5	0.84%	6	0.25%

3.3 Types of shots

Each game had a different total number of throws, and each throw was one of four different types discussed earlier. The four types of shots are hitting a rim or rims then landing the ball inside a cup, landing the ball inside a cup without hitting any rims, hitting a rim or rims then missing any cup, and missing any cup without hitting any rim. It was of interest to know which types of shots were more popular than others.

The results show that 82.49% of shots were direct misses, 3.93% were direct cups, 8.25% hit a rim and then missed, and 5.33% hit a rim and then landed in a cup. These results reflect proportions of games in general and do not necessarily reflect probabilities of these occurrences at given times in each game. They show, e.g., that overall, if a ball hit a rim, it will most likely not go into a cup. This statement, however, might be untrue at different times of the game. The results show that, overall, the probability of making a shot was about 9.26% while the probability of missing was 90.74%.

3.4 Types of games

Recalling from the previous section, there were two ways to categorize types of games. The two ways are grouped and labeled as "Types of Games I," and "Types of Games II."

3.4.1 Types of Games I: Chokes, landslide victories, and neck and neck matches

Types of Games I consists of "chokes," "landslides," and "necknecks." Although the term "choke" is used, it is merely used as a way to categorize games that would appear to have a player suffering from the psychological effect known as "choking." In fact, one of the purposes of the paper is to show that games often appear to be ones where a player chokes, but it might be simply a result of randomness.

How a game is categorized depends on what the absolute value of the maximum score difference was in the game, and the absolute value of the final score difference. It is of interest to know the proportion of games which had the player with a massive lead allow the opponent to catch up before the end of the game, the proportion of games which had the player with a massive lead win the game, and the proportion of games where there simply was no massive lead. The results of these statistics are shown in Figure 8.

3.4.2 Types of Games II: surprises, no surprises, and others

The second way to categorize games is by "surprises," "no surprises," and "others." Those depend on which player had the largest lead during the game and which player was the winner. A surprising game, i.e. a

"surprise," is one where the player with the largest lead in the game is not the player who won the game. A game which is not surprising, i.e. a "no surprise," is just the opposite. However, if both players had the largest lead at some point in the game, it constitutes an "other."

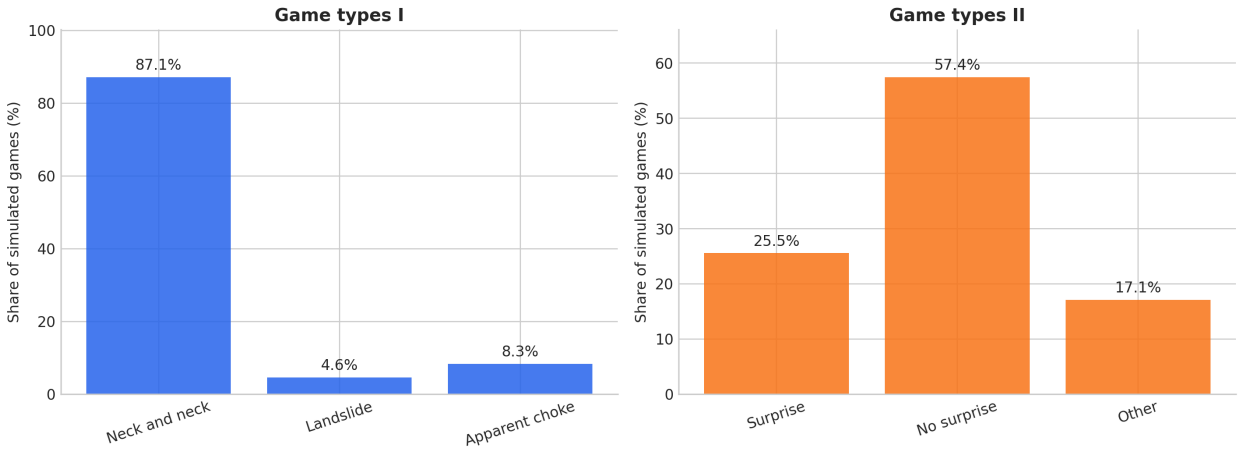

Figure 8: The proportions of the different types of games possible in a game of Beer Pong.

The results indicate the probability of what type of game will occur before the game starts at all. However, if one were to make an observation of a game mid-game and found that one player had a massive lead on another, the probability that the game consists of what would appear to be a choke is higher than the probability of the game ending with the player with the massive lead maintaining his/her massive lead, thus confirming the hypothesis. Out of all 10,000 games, 87.09% were neck and neck, 8.34% were apparent chokes, and 4.57% were landslides. Once it is known that a lead larger than 3 occurred, the proportion of apparent chokes is

$$\frac{0.0834}{0.0834 + 0.0457} = 0.6460. \tag{9}$$

Under Types of Games II, 25.53% were surprises, 57.40% were no surprises, and 17.07% were others.

3.5 P_t

P_t is the stochastic random variable defined as the number of points a given player has as a function of the number of times that player has thrown the ball, t. It is non-Markovian in nature since the probability of scoring a point depends on which cups have already been scored and removed, not merely the present score. However, a non-homogeneous Poisson process may be used as a model since the process contains occasional jumps of unity value whose frequency tends to decrease with time, thus having a time-varying intensity, $\lambda(t)$. A typical trajectory of P_t is shown in Figure 9.

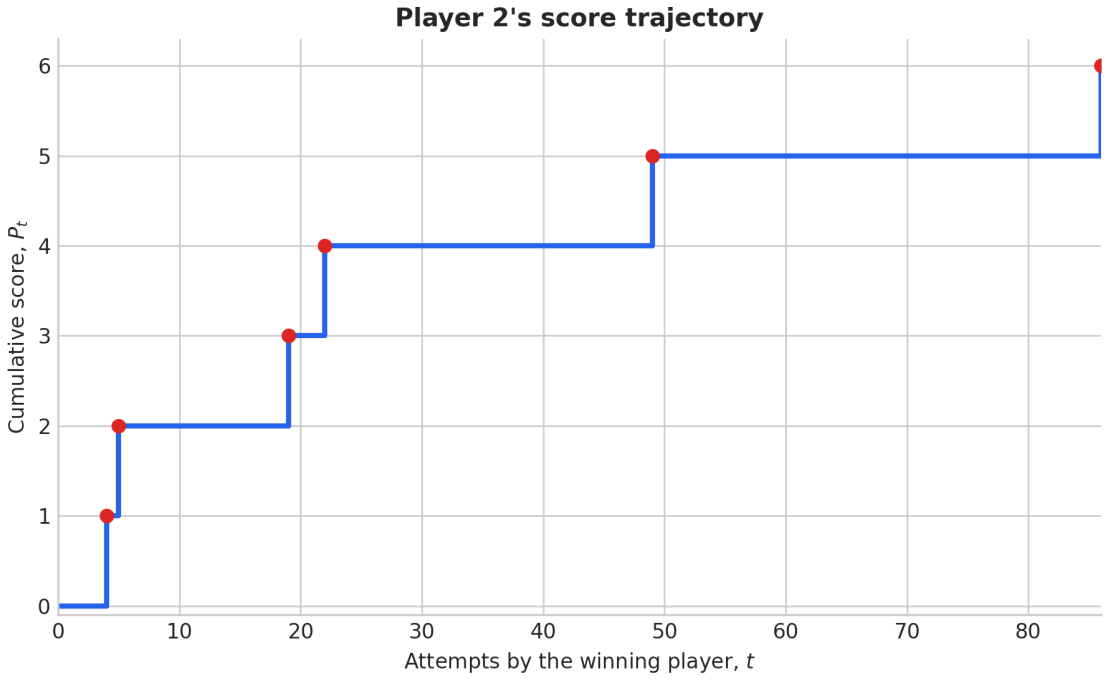

Figure 9: A typical trajectory of the stochastic process, P_t.

As discussed in Section 2.2.7, P_t was expected to increase by 1 point during certain time intervals. Playing 10,000 unconditional rack-clearance simulations revealed the six mean times in Table 1. Using Eqns. 1 and 2, an experimentally found $\lambda(t)$ was obtained. A plot of this $\lambda(t)$ and the smooth function obtained from it is seen in Figure 4.

Using Eqn. 1, the expectation value of P_t, i.e. $m(t)$, was found. This $m(t)$ indicates how many points a player is expected to have as a function of the number of times that player has thrown the ball, and is shown in Figure 5.

Substituting $m(t)$ into the Poisson probability formula gives 6 different probability distributions, $P(P_t = n)$ for $n = 1, 2, 3, 4, 5$, and 6. Those six probability distributions are depicted in Figure 6. Note that the six figures illustrate probability distributions and not probability density functions. Therefore, the probability itself is on the ordinate and is not to be taken as the integral over an interval of time of one of the functions.

Finally, the expected-score curve was compared with the mean score of 3,000 rack clearances. As shown in Figure 7, the curve closely followed the simulation and achieved $R^2 = 0.98$.

4 Conclusion

4.1 General remarks

The psychological phenomenon of choking may certainly play a contribution to the higher frequency of games of Beer Pong which consist of small final score discrepancies and large initial score discrepancies than large final score discrepancies and large initial score discrepancies. However, the results of this work suggest that this difference in frequencies does not need any psychological effects for its existence. What happened on the day of the ugly sweater party may have happened because the probability of making a shot decreases as the number of cups on the opponent's side diminishes. That is a simple consequence of the fact that the probability space changes. The probability is highly dependent on the location of the cups and whether or not the cups are near each other. The reason is due to the phenomenon of hitting rims of cups. The simulation may have placed a higher probability of landing a ball into a cup after hitting a rim than in real life, but this high probability may as well be compensated by the fact that real people aim. Aiming brings

the ball to the general location of the group of cups that are aggregated without necessarily landing the ball in the exactly desired position.

The probability of the outcome of a game is contingent upon information already gathered about the game. In the case of the observation made at the ugly sweater party, the score at some point in the game was known. According to the simulations, the probability that a game contains what looks like a choke before any other information is known is only about 8%. However, once it was observed that at some time there was a score discrepancy larger than 3, that information changed the probability. The probability that the game then ends in a small discrepancy score is about 65%. Having said that, there is still a larger probability that the game will end in a small discrepancy score than a large one. The probability of a "choke" is small, but once it is learned that a "neckneck" match is not possible, the probability of a "choke" changes dramatically.

The $\lambda(t)$ that speaks on all of the statistics of P_t was obtained experimentally from the simulations, but varies from person to person in real life. However, the general principle, i.e. the shape and general form of $\lambda(t)$, may be the same for many players. To obtain a real $\lambda(t)$, $m(t)$, and $P(P_t = n)$, real games would need to be observed and the constants C and r fit to the player or group in question. The strong fit to new simulations, $R^2 = 0.98$, shows that the smooth $m(t)$ does a good job of describing the average behavior of the simulator.

4.2 Suggestions for further research

The simulation has a lot of room for modification. For example, it would be interesting to know how the results compare if there were teams playing and some if not all of the team rules were applied such as two balls being thrown, the fact that both members of the team have to score the last point in the same round, etc.

It is also of interest to extrapolate the statistics gathered to higher numbers of cups without altering the simulation's algorithm. The $\lambda(t)$, $m(t)$, and $P(P_t = n)$ obtained from simulating the game with 6 cups should not be directly extended to 10 cups or more. The coefficients C and r would be different. It might be possible, however, to find a $C(n)$ and an $r(n)$ in order to generalize the model to n cups.

It would also be interesting to explore the properties of the difference in scores between two players, $P_{1t} - P_{2t}$. The two stochastic processes have equivalent rules and should have equivalent marginal intensities. Whether their difference behaves as a martingale under the turn and stopping rules would be an interesting question for further research.

References

[1] Sanjit Prasad. *Nonhomogeneous Poisson Processes.*

[2] Sheldon Ross. *Simulation.* Academic Press, fifth edition, 2012.

[3] Salih N. Neftci. *An Introduction to the Mathematics of Financial Derivatives.* Academic Press, first edition, 1996.